

MECHATRONICS AND IOT

ASSIGNMENT 2

Smart Inventory Management System for Construction Sites

TEAM MEMBERS:

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S - 24MER024

LOGESH S 24MER027

1. Project Overview

The **Smart Inventory Management System for Construction Sites** is an **IoT-based material monitoring and management system** developed to improve the way construction materials are stored, monitored, and utilized at construction sites.

Construction projects require a large quantity of materials such as **cement bags, steel rods, bricks, sand, pipes, electrical components, tools, and other equipment**. Managing these materials manually is difficult because the quantity of materials continuously changes whenever materials are received, moved, or used. Workers or supervisors normally maintain records using notebooks or spreadsheets. This method can result in **incorrect counting, delayed updates, material wastage, overstocking, shortage, loss, and even theft**.

To overcome these problems, the proposed system uses **IoT technology and sensors** to automatically monitor the quantity and movement of materials. An **ESP32 microcontroller** acts as the main controller of the system. It collects information from different sensors and processes the data to determine the current inventory condition.

2. Problem Statement

Construction sites face several inventory-related problems.

Existing problems

1. **Manual counting**
 - Workers usually count materials manually.
 - This takes time and effort.
2. **Incorrect inventory records**
 - Manual records can contain calculation or writing errors.
3. **Material shortage**
 - A material may become unavailable without the site manager knowing early.
4. **Material wastage**
 - Excess materials may be purchased and stored unnecessarily.
5. **Material loss**
 - Materials can be misplaced or moved to another location without proper records.

6. Unauthorized movement

- It may be difficult to identify who moved a particular material.

7. No real-time information

- The site manager may not know the current stock quantity immediately.

8. Poor resource utilization

- Materials may remain unused in one location while another location requires them.

3. Proposed Solution

The proposed system uses **IoT technology and sensors** to automatically monitor construction materials.

Main components

- **ESP32** – Main controller
- **RFID Reader** – Identifies materials/users
- **RFID Tags** – Unique identification for materials
- **Load Cell** – Measures material weight
- **HX711** – Connects the load cell to ESP32
- **Ultrasonic Sensor** – Measures material level in containers
- **OLED Display** – Displays inventory information
- **Buzzer** – Gives warning
- **LED** – Shows system status
- **Wi-Fi** – Sends information to the IoT dashboard

Example

Suppose the construction site has **cement**.

Initially:

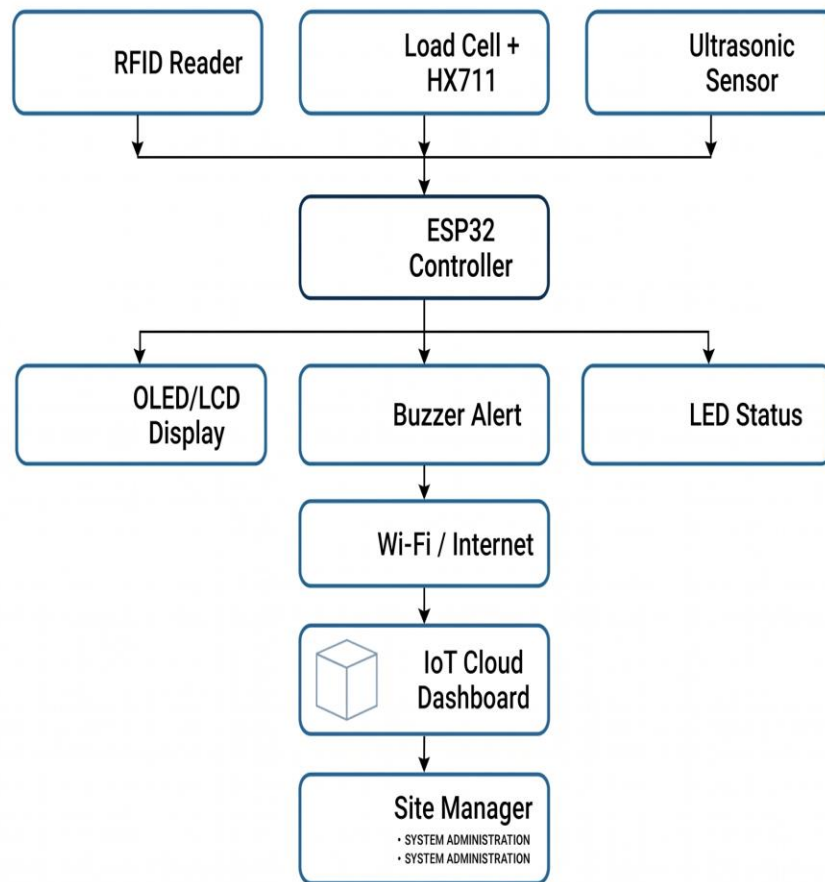
Cement = 10 kg

When 3 kg is used:

Remaining cement = 7 kg

The ESP32 automatically detects the new quantity and updates the dashboard.

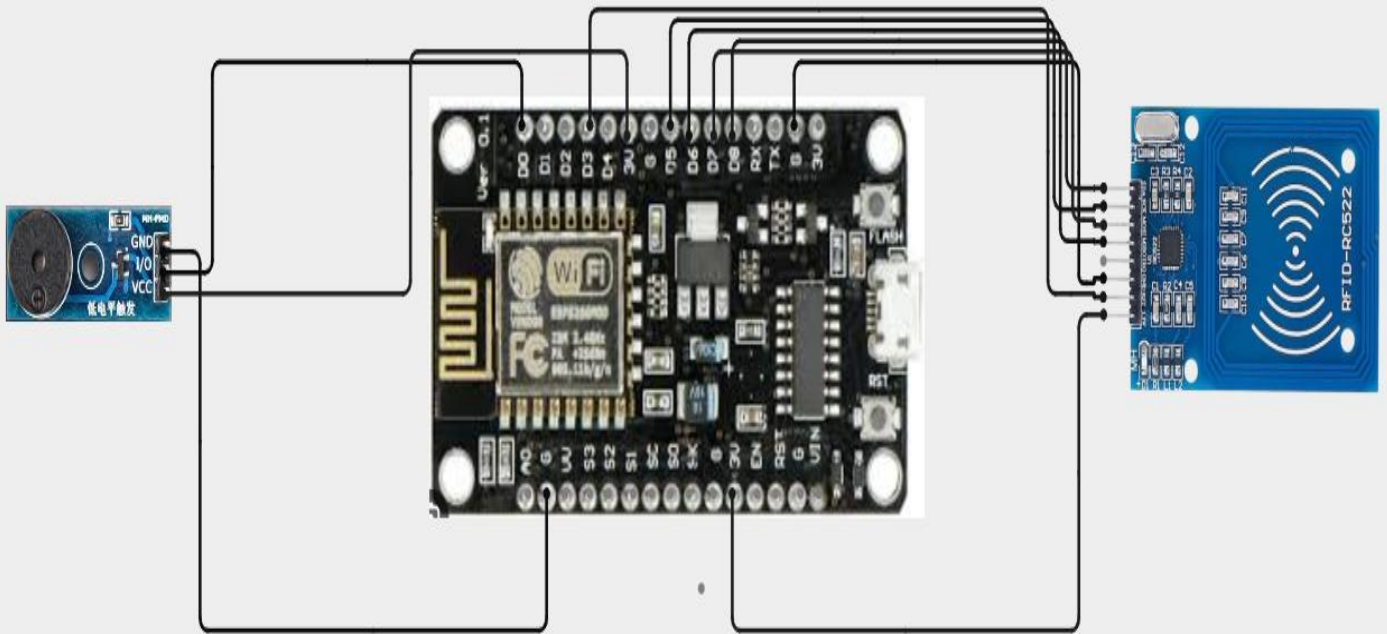
4. System Architecture



5. Circuit Table

Component	Sensor Pins	ESP32 Pins (GPIO)
RFID RC522	SDA, SCK, MOSI, MISO, RST	5, 18, 23, 19, 22
HX711 (Load Cell)	DT, SCK	16, 17
Ultrasonic Sensor	TRIG, ECHO	26, 27
OLED Display	SDA, SCL	21, 22
Buzzer	Positive (+)	25

6. Circuit Design



7. Algorithm

Step 1

Start the system.

Step 2

Initialize ESP32, RFID, load cell, ultrasonic sensor, OLED, and buzzer.

Step 3

Connect ESP32 to Wi-Fi.

Step 4

Check for an RFID tag.

Step 5

Identify the material or worker from the RFID UID.

Step 6

Read the load-cell value and calculate the material weight.

Step 7

Read the ultrasonic sensor and determine the material level.

Step 8

Update the inventory quantity.

Step 9

Compare the current stock with the minimum stock level.

Step 10

If the stock is below the minimum level, activate the buzzer.

Step 11

Display the inventory status on the OLED.

Step 12

Send the data to the IoT dashboard.

Step 13

Repeat the process continuously.

8. Coding:

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```

#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

// =====
// WIFI SETTINGS
// =====

const char* ssid = "Luky003";
const char* password = "kowshik03";

WiFiClient client;

// =====
// THINGSPEAK SETTINGS
// =====

unsigned long channelID = 3459014;

const char* writeAPIKey = "HPFMS1KQ7C86UZFC";

// =====
// RC522 RFID - ESP8266 GPIO PINS
// =====

#define SS_PIN 15    // D8
#define RST_PIN 0    // D3

MFRC522 mfrc522(SS_PIN, RST_PIN);

// =====
// I2C LCD - ESP8266 GPIO PINS
// =====

#define SDA_PIN 4    // D2
#define SCL_PIN 5    // D1

LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====
// BUZZER - LOW ACTIVE
// =====

#define BUZZER_PIN 16 // D0

```

```

// LOW = Buzzer ON
// HIGH = Buzzer OFF

// =====
// SETUP
// =====

void setup() {

    // -----
    // SERIAL MONITOR
    // -----

    Serial.begin(9600);
    delay(1000);

    Serial.println();
    Serial.println("=====");
    Serial.println("    RFID SYSTEM");
    Serial.println("=====");

    // =====
    // BUZZER SETUP
    // =====

    pinMode(BUZZER_PIN, OUTPUT);

    // LOW ACTIVE BUZZER
    // HIGH = OFF
    digitalWrite(BUZZER_PIN, HIGH);

    // =====
    // START LCD
    // =====

    Wire.begin(SDA_PIN, SCL_PIN);

    lcd.init();
    lcd.backlight();

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("RFID SYSTEM");

```



```
lcd.setCursor(0, 1);
lcd.print("Starting...");

// =====
// CONNECT TO WIFI
// =====

Serial.println();
Serial.println("Connecting to WiFi...");

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("Connecting WiFi");

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");
}

Serial.println();

Serial.println("WiFi Connected");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("WiFi Connected");

delay(1000);

// =====
// START THINGSPEAK
// =====

ThingSpeak.begin(client);

Serial.println("ThingSpeak Ready");
```

```

// =====
// START RFID
// =====

SPI.begin();

mfrc522.PCD_Init();

delay(1000);

// =====
// READY
// =====

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("RFID SYSTEM");

lcd.setCursor(0, 1);
lcd.print("Scan Your Card");

Serial.println("LCD Ready");
Serial.println("RC522 Ready");
Serial.println("ThingSpeak Ready");
Serial.println("Buzzer Ready");
Serial.println("Scan Your Card");

Serial.println("=====");
}

// =====
// LOOP
// =====

void loop() {

// -----
// CHECK FOR NEW RFID CARD
// -----

if (!mfrc522.PICC_IsNewCardPresent()) {
    return;

```

```
}
```

```
// -----  
// READ RFID CARD  
// -----
```

```
if (!mfrc522.PICC_ReadCardSerial()) {  
    return;  
}
```

```
// -----  
// CREATE UID STRING  
// -----
```

```
String uid = "";
```

```
Serial.print("UID: ");
```

```
for (byte i = 0; i < mfrc522.uid.size; i++) {
```

```
    if (mfrc522.uid.uidByte[i] < 0x10) {
```

```
        Serial.print("0");
```

```
        uid += "0";  
    }
```

```
    Serial.print(  
        mfrc522.uid.uidByte[i],  
        HEX  
    );
```

```
    uid += String(  
        mfrc522.uid.uidByte[i],  
        HEX  
    );  
}
```

```
uid.toUpperCase();
```

```
Serial.println();
```

```
// =====
```

```
// DOOR
// =====

if (uid == "BA8A8132") {

    showKnownTag(
        "Door",
        uid,
        1
    );
}

// =====
// LAB
// =====

else if (uid == "7BABD13E") {

    showKnownTag(
        "Lab",
        uid,
        2
    );
}

// =====
// SATHEESH
// =====

else if (uid == "D61A9269") {

    showKnownTag(
        "Satheesh",
        uid,
        3
    );
}

// =====
// =====

else if (uid == "6F22CB48") {

    showKnownTag(
        "Sabeer",
```

```

        uid,
        4
    );
}

// =====
// =====

else if (uid == "1F22EB48") {

    showKnownTag(
        "Prasana",
        uid,
        5
    );
}

// =====
// UNKNOWN CARD
// =====

else {

    showUnknownTag(uid);
}

// -----
// STOP RFID COMMUNICATION
// -----

mfr522.PICC_HaltA();

mfr522.PCD_StopCrypto1();

// -----
// READY FOR NEXT CARD
// -----

delay(1000);

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("RFID SYSTEM");

```

```

    lcd.setCursor(0, 1);
    lcd.print("Scan Your Card");
}

// =====
// FUNCTION: KNOWN RFID TAG
// =====

void showKnownTag(
    String name,
    String uid,
    int personID
) {

    // -----
    // BUZZER OFF
    // LOW ACTIVE BUZZER
    // HIGH = OFF
    // -----

    digitalWrite(
        BUZZER_PIN,
        HIGH
    );

    // -----
    // SERIAL OUTPUT
    // -----

    Serial.println();

    Serial.print("Known RFID Tag: ");
    Serial.println(name);

    Serial.print("UID: ");
    Serial.println(uid);

    Serial.println("Access Granted");

    Serial.println("-----");

    // -----
    // LCD OUTPUT
    // -----

```

```

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("Access Granted");

lcd.setCursor(0, 1);
lcd.print(name);


// -----
// SEND DATA TO THINGSPEAK
// -----

sendToThingSpeak(
    uid,
    personID,
    1
);


// -----
// KEEP RESULT ON LCD
// -----

delay(3000);


// -----
// MAKE SURE BUZZER IS OFF
// -----

digitalWrite(
    BUZZER_PIN,
    HIGH
);
}


// =====
// FUNCTION: UNKNOWN RFID TAG
// =====

void showUnknownTag(
    String uid
){

    // -----

```

```
// SERIAL OUTPUT
// -----

Serial.println();

Serial.print("Unknown RFID Tag: ");
Serial.println(uid);

Serial.println("Access Denied");

Serial.println("-----");

// -----
// LCD OUTPUT
// -----

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("Unknown Tag");

lcd.setCursor(0, 1);
lcd.print("Access Denied");

// -----
// SEND DATA TO THINGSPEAK
// -----

sendToThingSpeak(
  uid,
  0,
  0
);

// -----
// BUZZER - LOW ACTIVE
// 3 BEEPS
// -----

for (int i = 0; i < 3; i++) {

  // BUZZER ON
  digitalWrite(
    BUZZER_PIN,
    LOW
```



```

);

delay(300);

// BUZZER OFF
digitalWrite(
  BUZZER_PIN,
  HIGH
);

delay(300);
}

// -----
// MAKE SURE BUZZER IS OFF
// -----

digitalWrite(
  BUZZER_PIN,
  HIGH
);

// -----
// KEEP RESULT ON LCD
// -----

delay(2000);
}

// =====
// FUNCTION: SEND DATA TO THINGSPEAK
// =====

void sendToThingSpeak(
  String uid,
  int personID,
  int accessStatus
) {

  // -----
  // CHECK WIFI
  // -----

  if (WiFi.status() != WL_CONNECTED) {

```

```
Serial.println(
  "WiFi Disconnected"
);

return;
}

// -----
// FIELD 1 = RFID UID
// -----

ThingSpeak.setField(
  1,
  uid
);

// -----
// FIELD 2 = PERSON ID
// -----

ThingSpeak.setField(
  2,
  personID
);

// -----
// FIELD 3 = ACCESS STATUS
// -----

ThingSpeak.setField(
  3,
  accessStatus
);

// -----
// WRITE DATA TO THINGSPEAK
// -----

int response = ThingSpeak.writeFields(
  channelID,
  writeAPIKey
);
```

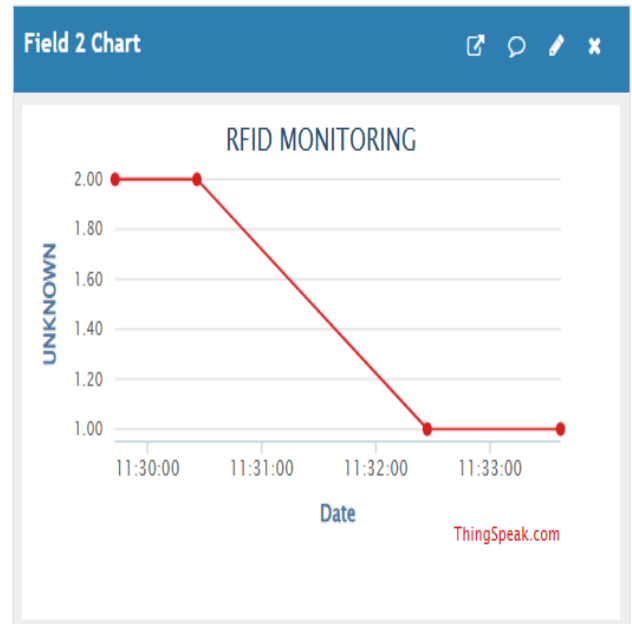
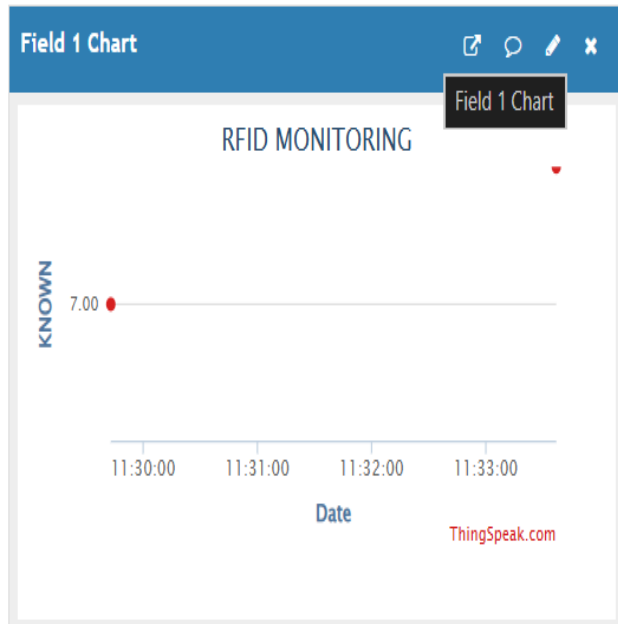
```
// -----  
// CHECK RESPONSE  
// -----  
  
if (response == 200) {  
  
    Serial.println(  
        "ThingSpeak Update Successful"  
    );  
  
}  
  
else {  
  
    Serial.print(  
        "ThingSpeak Error: "  
    );  
  
    Serial.println(response);  
}  
}
```

9. System Working

Channel Stats

Created: 9 minutes ago

Entries: 5



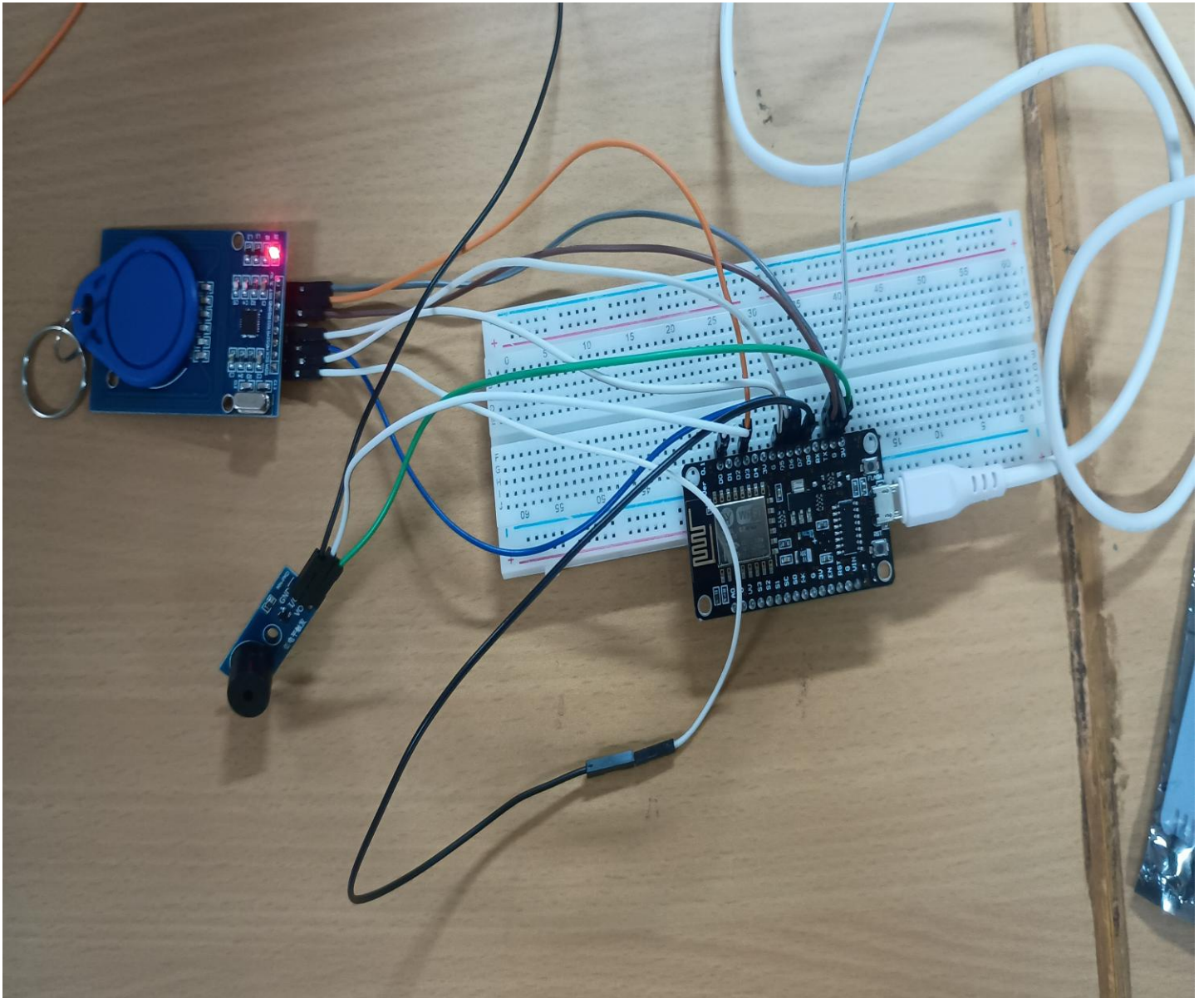
Working sequence

Data Sensing (RFID, Load Cell, Ultrasonic) → Processing & Verification (ESP32) → Local Display & Alerts (OLED, Buzzer, LED) → Wi-Fi / MQTT Transmission → Cloud Storage & Dashboard Visualization (ThingSpeak) → Remote Monitoring & Data Analysis

10. Bill of Materials

S.No	Component	Qty.	Approx. Cost
1	ESP32	1	₹350–500
2	RFID RC522	1	₹100–180
3	RFID Tags	3	₹30–60
4	Load Cell	1	₹150–300
5	HX711	1	₹50–100
6	Ultrasonic Sensor	1	₹50–100
7	OLED Display	1	₹100–180
8	Buzzer	1	₹10–30
9	Breadboard	1	₹80–150
10	Jumper Wires	1 set	₹50–100
11	USB/Power Supply	1	₹100–200
12	Storage Container	1	₹100–200

11. Prototype Implementation



Implementation steps

- Fix the load cell below the material platform.
- Connect the load cell to HX711.
- Place the ultrasonic sensor at the top of the container.
- Place the RFID reader near the material entry/exit point.
- Connect all sensors to ESP32.
- Upload the program.
- Calibrate the load cell using a known weight.
- Connect ESP32 to Wi-Fi.
- Test RFID detection.

- Add and remove material to test inventory changes.
- Test the low-stock buzzer.
- Display the results on the OLED and IoT dashboard.

12. Results and Performance Analysis

Performance

- Provides **real-time monitoring** of construction materials.
- Load cell provides **accurate weight measurement** after calibration.
- RFID provides **quick material/user identification**.
- Ultrasonic sensor monitors **material level continuously**.
- ESP32 updates inventory data automatically.
- Buzzer provides an **immediate low-stock alert**.
- OLED displays **current material quantity and status**.
- IoT dashboard enables **remote inventory monitoring**.

Advantages

- Reduces manual inventory checking.
- Saves time and labor.
- Provides real-time inventory information.
- Helps prevent material shortages.
- Reduces inventory recording errors.
- Tracks material movement using RFID.
- Provides low-stock alerts.
- Allows remote monitoring through IoT.
- Can be expanded to multiple materials.
- Low-cost and suitable for prototype development.

Disadvantages

- Sensors require proper calibration.
- RFID has limited reading range.
- Load-cell accuracy depends on installation.
- Ultrasonic measurement can be affected by irregular material surfaces.
- Wi-Fi is required for remote monitoring.
- Electronic components need protection from construction-site dust and moisture.

